



Tribhuvan University

Faculty of Humanities and Social Sciences

ROAD SENSE: ALTERNATIVE ROUTE FINDER

A PROJECT REPORT

Submitted to

Department of Computer Application

D.A.V. College

In partial fulfillment of the requirements for the Bachelors in Computer Application

Submitted by

Anish Dangol

[TU Reg. No: 7-2-920-62-2020]

Shrawan, 2082

Under the Supervision of

Er. Pralhad Chapagain



Tribhuvan University

Faculty of Humanities and Social Sciences

D.A.V. College

Supervisor's Recommendation

I hereby recommend that this project prepared under my supervision by **ANISH DANGOL** entitled “**ROAD SENSE: ALTERNATIVE ROUTE FINDER**” in partial fulfillment of the requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

Er. Pralhad Chapagain

SUPERVISOR

Lecturer

Department of Computer Application

Jawalakhel, Lalitpur, Nepal



Tribhuvan University

Faculty of Humanities and Social Sciences

D.A.V. College

LETTER OF APPROVAL

This is to certify that this project prepared by **ANISH DANGOL** entitled “**ROAD SENSE: ALTERNATIVE ROUTE FINDER**” in partial fulfillment of the requirements for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

Er. Pralhad Chapagain Department of Computer Application Jawalakhel, Lalitpur	Mr. Sudip Adhikari, Program Head Department of Computer Application Jawalakhel, Lalitpur
Internal Examiner	External Examiner

Abstract

With the growing complexity of urban traffic in Kathmandu, particularly along the Ring Road, this project addresses the limitations of traditional navigation systems that fail to account for real-time, user-reported road conditions. The system introduced in this study incorporates a mobile-based navigation platform leveraging the Bidirectional A* algorithm, supported by real-time obstacle reporting from users. Through features such as obstacle creation, route rerouting, and user authentication, the application offers a responsive navigation experience tailored to the dynamic traffic environment of Kathmandu and Lalitpur. The development process adopted the Incremental Model, allowing phased implementation and refinement across core modules including navigation, obstacle detection, alternative route generation, and user validation. The system emphasizes practical usability through map interaction, profile-based obstacle rating, and search functionalities. Testing results show promise in enhancing route accuracy and adaptability. While limited geographically and still pending real-time navigation and notification features, the system lays a strong foundation for crowd-informed urban mobility solutions.

Keywords: *Real-time navigation, Bidirectional A* algorithm, traffic congestion, user-reported obstacles, obstacle detection*

Acknowledgement

I would like to express my sincere gratitude to my supervisor, Er. Pralhad Chapagain, whose guidance, feedback, and encouragement played a crucial role throughout the development of this project. His insights and constructive suggestions were invaluable in overcoming challenges and shaping the final outcome of this work.

I am also thankful to my colleagues and project team members for their constant support, idea sharing, and collaboration during every phase of this project. The teamwork and shared effort made the process both productive and rewarding.

I would also like to extend my appreciation to D.A.V. College, the faculty members, and the Department of Computer Application for providing the necessary resources, academic environment, and technical assistance that enabled the successful completion of this project.

Lastly, I am grateful to everyone who contributed directly or indirectly to the realization of this work.

Regards,

Anish Dangol

Table of Contents

SUPERVISOR'S RECOMMENDATION	i
LETTER OF APPROVAL	ii
ABSTRACT	iii
ACKNOWLEDGEMENT	iv
LIST OF ABBREVIATIONS	vii
LIST OF FIGURES	viii
LIST OF TABLES	ix
CHAPTER 1: INTRODUCTION.....	1
1.1 Introduction.....	1
1.2 Problem Statement.....	2
1.3 Objectives.....	3
1.4 Scopes and Limitations.....	3
1.5 Development Methodology.....	3
1.6 Report Organization	5
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW.....	6
2.1 Background Study	6
2.2 Literature Review	7
CHAPTER 3: SYSTEM DESIGN AND ANALYSIS.....	10
3.1 System Analysis	10
3.1.1 Requirement Analysis	10
3.1.2 Feasibility Analysis	12
3.1.3 Data Modeling: ER Diagram	14
3.1.4 Process Modeling: DFD	15
3.2 System Design.....	17
3.2.1 Architectural Design	17
3.2.2 Database Schema Design	20
3.2.3 Interface Design	21
3.3 Algorithm Details	24
CHAPTER 4: IMPLEMENTATION AND TESTING.....	28
4.1 Implementation	28

4.1.1 Tools Used	28
4.1.2 Implementation Details of Modules	28
4.2 Testing	29
4.2.1 Test Cases for Unit Testing	29
4.2.1 Test Cases for System Testing	34
CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS	35
5.1 Conclusion	35
5.2 Lessons Learnt / Outcome.....	35
5.3 Future Recommendations	36
REFERENCES.....	38
APPENDICES	40

List of Abbreviations

A*	A-Star (Pathfinding Algorithm)
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DFD	Data Flow Diagram
ER	Entity-Relationship
GPS	Global Positioning System
RTDB	Real-Time Database
UAT	User Acceptance Testing
UI	User Interface

List of Figures

Figure 1.1 Incremental Model of Development of RoadSense.....	4
Figure 3.2 Gantt Chart of RoadSense	13
Figure 3.3 Entity Relationship Diagram of RoadSense	14
Figure 3.4 Data Flow Diagram Level 0 of RoadSense	15
Figure 3.5 Data Flow Diagram Level 1 of RoadSense	16
Figure 3.6 Architectural Design of RoadSense.....	17
Figure 3.7 Working flow of the Bidirectional A* Algorithm.....	18
Figure 3.8 Schema Diagram of RoadSense	20

List of Tables

Table 4.1 User Authentication Test Case	30
Table 4.2 Map View Test Case.....	30
Table 4.3 Obstacle Verification Test Case.....	31
Table 4.4 Obstacle Report Test Case	32
Table 4.5 Algorithm Test Case.....	32
Table 4.6 System Test Case.....	34

CHAPTER 1: INTRODUCTION

1.1 Introduction

Urban transportation is a fundamental aspect of modern cities, influencing economic productivity, environmental sustainability, and the overall quality of life. As urban populations continue to rise, road networks in many cities face significant challenges due to congestion, inefficient traffic management, and increasing vehicle numbers. Traffic congestion leads to lost productivity, higher fuel consumption, and increased air pollution, making it a critical issue for urban planners and policymakers. Various studies have explored solutions such as intelligent traffic management systems, public transportation improvements, and urban road expansion to mitigate these challenges. However, each city has unique infrastructural and demographic characteristics that require tailored approaches to traffic optimization [1].

This project focuses on Kathmandu and Lalitpur area, a crucial transportation corridor that facilitates mobility within the city and its surrounding areas. With rapid urbanization and rising vehicle ownership, traffic congestion along this road has become a pressing concern, leading to delays, increased fuel consumption and even accidents. Despite ongoing infrastructure developments, such as road widening and intersection modifications, congestion remains a persistent issue due to inconsistent enforcement of traffic regulations, lack of pedestrian-friendly infrastructure, and inadequate public transport systems. Understanding traffic flow patterns, congestion hotspots, and commuter behavior is essential for formulating effective strategies to enhance road efficiency and urban mobility.

To address these challenges, this project relies on user-reported congestion data to analyze traffic conditions along the Kathmandu Ring Road. The study gathers real-time insights from commuters and road users, identifying congestion hotspots based on firsthand experiences. Factors such as vehicle density, peak-hour delays, and bottleneck locations are examined through direct observations and reported data. Potential solutions, including better traffic management policies, optimized road use, and improved pedestrian infrastructure, are assessed based on feasibility and impact [2]. Additionally, lessons from cities facing similar urban mobility challenges are considered to develop recommendations that align with Kathmandu's specific traffic dynamics. By focusing on real-world user input, this research aims to offer practical solutions grounded in the lived experiences of daily commuters.

By concentrating on the Kathmandu Ring Road, this project contributes to the broader understanding of traffic congestion in rapidly growing cities with similar infrastructural and economic conditions. The findings can assist policymakers and urban planners in making informed decisions to enhance road efficiency, reduce traffic delays, and improve air quality [3]. If effectively implemented, the proposed recommendations can lead to a more sustainable and well-organized transportation network, benefiting not only daily commuters but also businesses and the environment.

1.2 Problem Statement

Traffic congestion is a major challenge in urban mobility, leading to increased travel times, fuel consumption, and environmental pollution. Many existing navigation systems provide shortest-path routing but fail to account for real-time road conditions such as accidents, weather disruptions, road maintenance, or other obstacles that impact traffic flow. Traditional navigation tools, such as Google Maps, primarily focus on static traffic data or estimated congestion levels, lacking the ability to incorporate dynamic obstacles reported by commuters themselves. This results in inefficient route planning, where users are often directed onto roads with unforeseen blockages, causing further delays. While studies have explored advanced algorithms for shortest-path calculations and congestion reduction, most rely on predictive models rather than user-reported real-time data [4].

This project aims to address these gaps by developing a mobile navigation system focused on real-time, user-reported road obstacles. By allowing users to flag issues such as accidents, weather hazards, and roadblocks, the system dynamically suggests alternative routes using the bidirectional A* algorithm. The initial implementation is scoped to Kathmandu's Ring Road, an area experiencing severe congestion due to rapid urbanization and infrastructure limitations. Similar studies have highlighted the effectiveness of integrating real-time data into navigation systems to enhance urban mobility [5]. By leveraging direct commuter input, this project provides a more responsive and adaptive navigation experience, reducing travel delays and improving road efficiency. The findings can contribute to smarter traffic management solutions for cities facing similar transportation challenges.

1.3 Objectives

- To enhance real-time navigation with user-reported road obstacles and alternative route recommendations.

1.4 Scopes and Limitations

Scopes

- Provides a medium to navigate through the shortest path from source to destination.
- Search navigation that makes panning to different areas of map view easier.
- Panning of map view to the current location.
- Obstacle creation with detailed information.
- Rating of obstacles authenticity which reflects on user's profile.

Limitations

- Limited to very specific geographic location, which is Kathmandu and Lalitpur.
- Alternative route may not always be the shortest path from source to destination.
- Ratings from other users for obstacle authenticity may not always be trustworthy.
- Doesn't include any component to prevent false report of obstacles due to the vague obstacle verification.

1.5 Development Methodology

The Incremental Model is suitable for developing the Bidirectional A* algorithm because it allows step-by-step implementation, testing, and optimization. The project can be started with a basic A* search, then extend it to bidirectional search, followed by performance optimizations such as efficient heuristics and data structures. This approach ensures early testing and validation, reducing the risk of errors while allowing flexibility for improvements. Additionally, each increment builds upon the previous one, making debugging and enhancements more manageable, leading to a more efficient and scalable pathfinding solution.

Build 1: Navigation System

This module provides core functionality for route navigation within the app. It calculates and displays paths from a user's current location to a chosen destination. The system incorporates map rendering and real-time location tracking to guide users effectively.

Build 2: User Authentication

This feature ensures secure access to the app by enabling user registration, login, and session management. It supports authentication methods such as email/password or social login, verifying users and managing access to personalized features.

Build 3: Obstacle Report

This allows users to report obstacles encountered on their route, such as construction zones, roadblocks, or inaccessible areas. The data collected is stored and visualized on the map to inform other users, enhancing overall route awareness.

Build 4: Alternative Route Finder

If obstacles are detected on the original route, this system calculates alternative paths to help users reach their destinations efficiently. It considers obstacle data and real-time conditions to find optimal detours.

Build 5: User Report Rating

To ensure reliability of reported data, users can rate obstacle reports submitted by others. Highly rated reports are prioritized for visibility, while low-rated or outdated reports may be flagged for review, improving overall data quality.

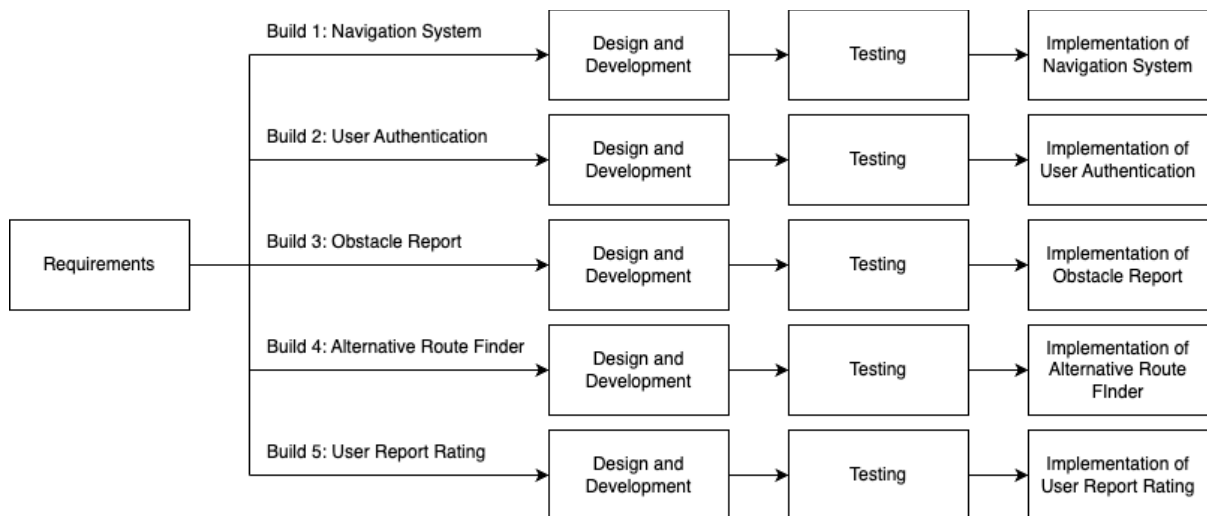


Figure 1.1 Incremental Model of Development of RoadSense

1.6 Report Organization

The first chapter of the documentation contains the introduction of the system, the reason for the need of this system, what this system aims to achieve, what scope this system covers and what limitation this system possesses. The second chapter of the documentation covers the background system of various existing system in the market, how they operate and how they are lacking. It also covers the literature review of various different projects and systems that are similar to this system, also research work done on such similar system. The third chapter of the documentation includes the analysis of the system through requirement, feasibility and the design of the system. The requirement analysis of the system covers the functional and the non-functional requirements that the system contains, whereas the feasibility analysis covers technical, operational, economic, and schedule feasibility of the system. This chapter also contains the core system design that the system is built on top of. The fourth chapter of the documentation covers about how the system was implemented and how it works in overall. It also talks about tools that are used to implement this system and how they were used to create this system.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Traffic congestion is a major issue in urban transportation, leading to delays, fuel wastage, and increased pollution. Traditional navigation systems provide route guidance but often fail to adapt to real-time road conditions such as accidents, construction, or extreme weather. Several studies have explored intelligent traffic management solutions to enhance urban mobility by integrating real-time data, predictive algorithms, and user-generated inputs, propose a real-time self-adaptive traffic management system that dynamically calculates the shortest routes based on road conditions [4]. Their approach relies on structured traffic data collected from taxi drivers, providing insights into congestion patterns and optimal paths [4]. While effective, this system is limited to predefined datasets and lacks direct user input on temporary roadblocks such as accidents or demonstrations. Their study focuses on algorithmic efficiency rather than incorporating real-time, user-reported data, which can provide a more responsive solution to sudden road obstacles.

Existing navigation apps like Google Maps, Waze, and HERE WeGo integrate real-time traffic updates, but they have limitations. Waze primarily relies on crowdsourced reports, which can sometimes be inaccurate or delayed, while Google Maps predicts congestion using GPS tracking and AI-driven models but provides limited obstacle reporting [3]. HERE WeGo offers reliable navigation with strong offline functionality but lacks extensive user-generated traffic data. Studies suggest that a hybrid approach—combining user-reported incidents with intelligent rerouting algorithms can improve navigation efficiency in dynamic urban environments.

This project builds upon these findings by developing a real-time, user-driven navigation system that allows commuters to report obstacles directly, enabling dynamic rerouting. Unlike systems that depend solely on pre-recorded data or automated detection, this approach ensures that sudden disruptions such as roadblocks, accidents, or weather hazards—are accounted for immediately. By focusing on Kathmandu's Ring Road, a high-traffic area with frequent road disruptions, this research aims to bridge the gap between existing navigation solutions and real-time user input, ultimately improving urban mobility and travel efficiency.

2.2 Literature Review

This part briefly describes the research study of related routing and navigation applications. The first study emphasized on the biggest navigation platform which is Google and how they implement navigation system. The second study aims to understand how the implementation for real time traffic awareness can be done through Waze. The third study about HERE WeGo is to analyze the possibilities for offline navigation capabilities and possible different travel modes. The fourth study mainly focuses on the existing navigation application of Nepal i.e., Galli Maps and how they collected and worked with local data and information about Nepal.

1. Google Maps

Google Maps is a comprehensive navigation platform that integrates GPS tracking and artificial intelligence to deliver accurate, real-time routing and traffic analysis. It draws data from a vast network of smartphone users and transport agencies, making it a globally trusted tool for daily commuting and long-distance travel [2].

Key Features:

1. Real-time traffic updates based on GPS data
2. Turn-by-turn voice navigation
3. Street View and indoor maps
4. Offline map access
5. Multimodal route planning (bus, car, walk, bike, ride-share)

Research Gap and Project Solution:

Although Google Maps provides estimated traffic insights and predictive routing, it lacks the ability to process real-time, user-reported road obstacles such as construction, protests, or temporary closures. This limitation reduces its responsiveness in rapidly changing urban environments. Our project addresses this by enabling users to directly report road obstacles, which are then verified through a community-based rating system. Using a bidirectional A* algorithm, the system recalculates paths in real time, avoiding reported obstacles and delivering more accurate navigation tailored for congested areas like Kathmandu's Ring Road [6].

2. Waze

Waze is a crowd-powered navigation app known for its real-time responsiveness to road conditions. It relies heavily on user contributions to detect and report traffic issues, making it agile in adjusting routes based on sudden disruptions [7].

Key Features:

1. User-reported traffic incidents (e.g., accidents, hazards)
2. Dynamic rerouting based on real-time conditions
3. Real-time hazard and police alerts
4. Scheduled trip planning and departure suggestions
5. Community map editing tools

Research Gap and Project Solution:

While Waze is highly interactive, it struggles with data authenticity due to the lack of a structured validation process. Inaccurate or outdated user reports can affect routing decisions and lead to misinformation. Our project solves this by implementing a trust-based system where obstacle reports are rated by other users, boosting or suppressing their visibility based on reliability. Only credible data influences the rerouting process, which is handled by a bidirectional A* algorithm that intelligently avoids verified obstacles, enhancing both trust and efficiency in navigation.

3. HERE WeGo

HERE WeGo is a navigation application designed for reliability and strong offline functionality. It primarily uses data from official traffic sources such as government and transportation agencies, ensuring a high level of data accuracy even without user participation [8].

Key Features:

1. Full offline map downloads
2. Real-time traffic updates from official sources
3. Multimodal travel support
4. In-car system integration
5. Fare estimation for taxis and ride-share

Research Gap and Project Solution:

HERE WeGo excels in offline access but lacks the flexibility to respond to temporary, ground-level obstacles that are not captured by official data sources. This makes it less suitable for areas with frequent, unreported disruptions. Our project fills this gap by combining offline map access with real-time user-generated obstacle data. Users can report issues which are dynamically incorporated into route planning, even when internet connectivity is intermittent. The system merges the reliability of offline navigation with the responsiveness of community-driven reporting, ensuring adaptable routing in unpredictable conditions.

4. GalliMaps

GalliMaps is a Nepalese navigation application tailored for the country's complex and narrow urban streets ("gallis"). It focuses on detailed local mapping, which is often lacking in global navigation tools, making it especially useful for deliveries and urban navigation in Nepal [9].

Key Features:

1. Detailed maps of small streets and alleys
2. House number-based search functionality
3. 360-degree street views
4. Real-time GPS-based location tracking
5. Offline navigation support
6. User-contributed map updates

Research Gap and Project Solution:

While GalliMaps provides excellent coverage of Nepal's local roadways, it lacks real-time route adjustment and obstacle-aware navigation. It does not offer dynamic rerouting based on temporary traffic conditions or disruptions. Our project extends this local precision by introducing real-time obstacle reporting and avoidance. Through a bidirectional A* algorithm with obstacle-avoidance heuristics, the system dynamically identifies alternative routes. A rating system ensures that reported issues are verified by the community, improving both accuracy and trust. This enables more efficient and responsive navigation within Kathmandu's dynamic and often unpredictable road environment.

CHAPTER 3: SYSTEM DESIGN AND ANALYSIS

3.1 System Analysis

Some requirements for this project were clear from the beginning, while others evolved during development, however the rest of the requirements were not as clear. The features and implementation were not clearly defined for the later phases for this project. Hence, Incremental Model of system development was used for this project as the project has the major requirements of the system clearly defined, however, some of the details of the project may evolve over time. The incremental model allows to clearly report and track the project's progress on each implementation stage. This also allows the project to receive feedbacks from supervisors on each implementation. With the above reason, incremental model of software development was used for this project.

3.1.1 Requirement Analysis

- i. Functional Requirements
 - Users can create accounts, log in, and manage profiles.
 - The system provides turn-by-turn navigation and reroutes based on obstacles.
 - Users can report accidents, roadblocks, and other traffic issues.
 - The map displays roads, landmarks, and real-time traffic updates.
 - Users receive alerts about road conditions and alternative routes.
 - Users can rate the reports given by other users.



Figure 3.1 Use Case Diagram of RoadSense

ii. Non-Functional Requirements

- Security
- User friendly Interface
- Scalability

3.1.2 Feasibility Analysis

i. Technical Feasibility

The project is going to be built with React Native for cross-platform compatibility. Firebase is chosen as the database of the system due to its real-time data synchronization, essential for instant obstacle reporting and route updates. MapLibre ensures cost-effective, high-performance map rendering. The bidirectional A algorithm* optimizes navigation by dynamically adjusting routes. This setup guarantees a smooth, scalable, and responsive system.

ii. Operational Feasibility

The project features an intuitive UI for reporting obstacles, viewing live updates, and receiving optimized routes. Firebase's real-time capabilities ensure instant data sync, enhancing user experience. MapLibre's customizable features allow seamless integration into the app. A moderation system may validate reports to maintain accuracy. The combination of React Native, Firebase, and MapLibre ensures efficient app operation with minimal maintenance.

iii. Economic Feasibility

Firebase's free-tier services significantly reduce backend development costs while ensuring seamless authentication, real-time data handling, and cloud functions. MapLibre eliminates expensive map licensing fees. React Native accelerates development by supporting both iOS and Android with a single codebase. These choices help development process, making the project cost-effective while maintaining high performance.

iv. Schedule Feasibility

The time schedule for different modules of this project can be seen as below in the following Gantt chart.

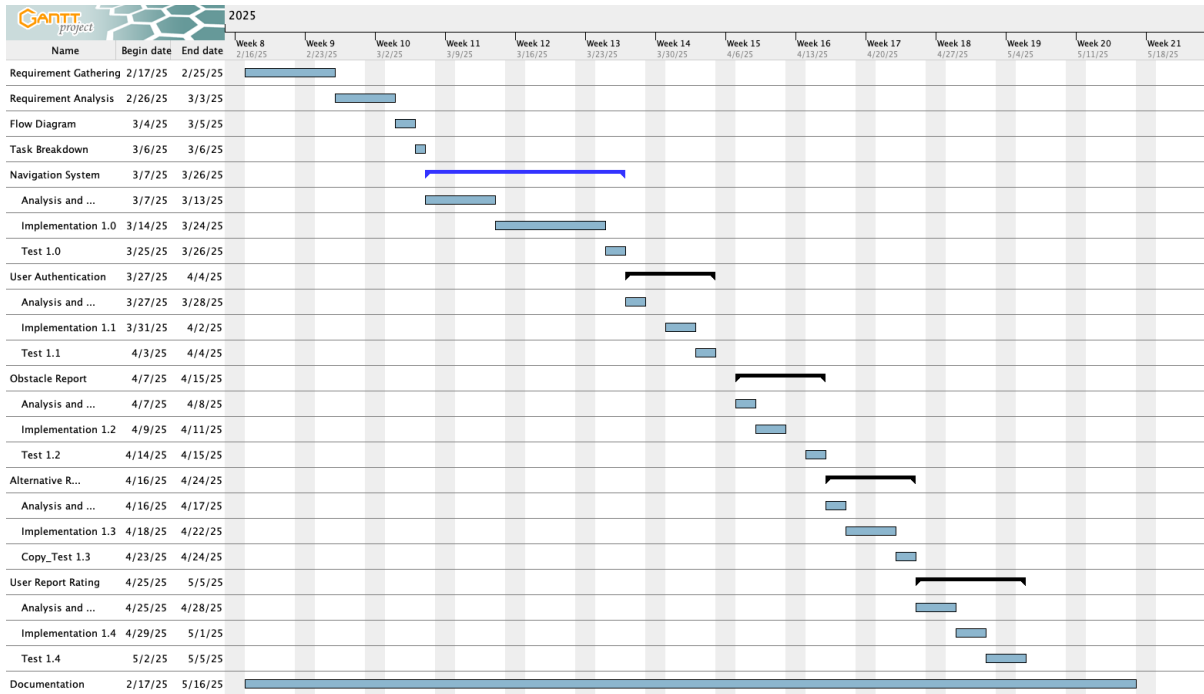


Figure 3.2 Gantt Chart of RoadSense

The Gantt chart for this project is shown in the above figure. The project is divided into five different modules. Since the project is based on incremental model, each module is individually implemented and are refined as per the requirement changes or needs.

3.1.3 Data Modeling: ER Diagram

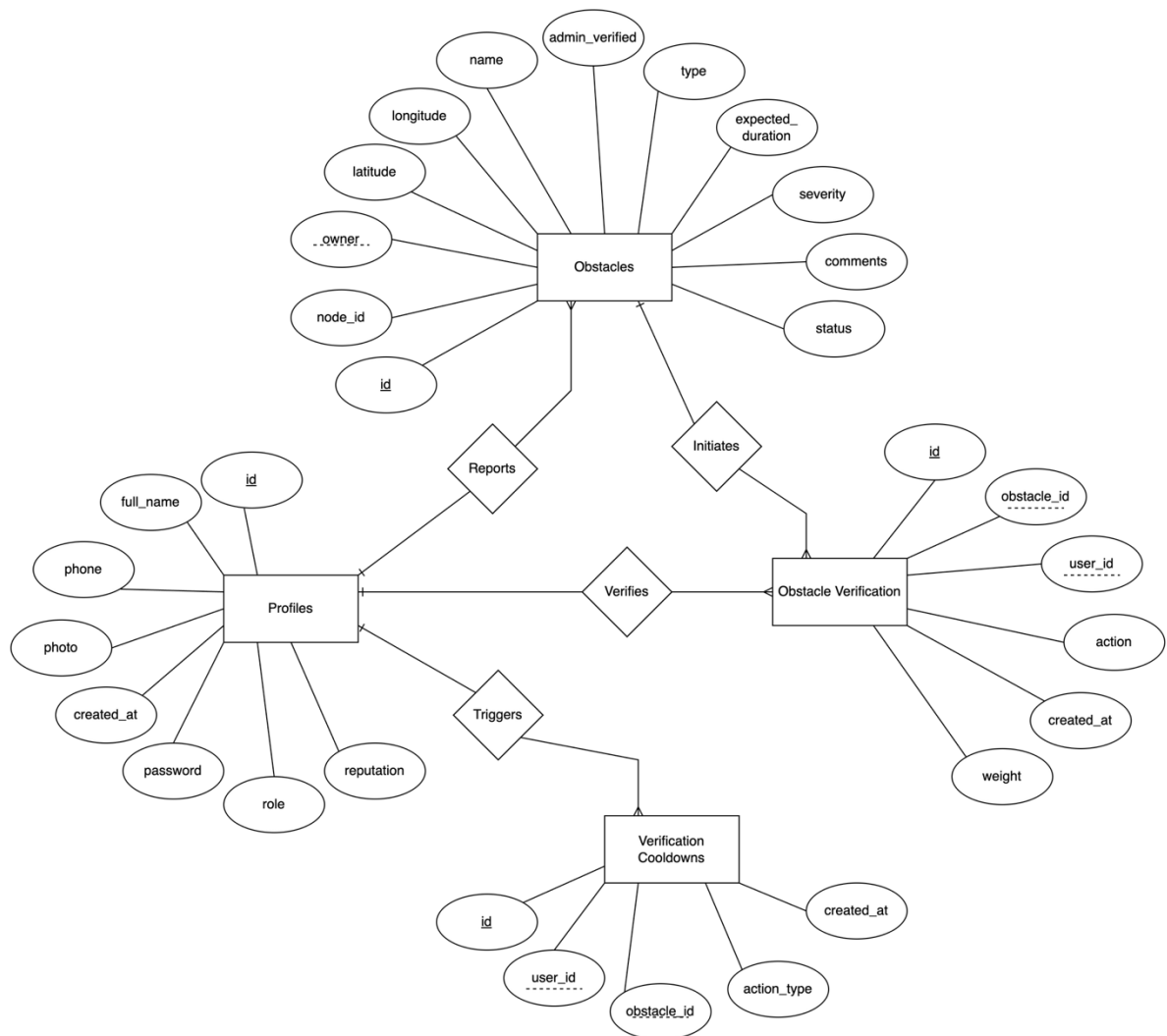


Figure 3.3 Entity Relationship Diagram of RoadSense

3.1.4 Process Modeling: DFD

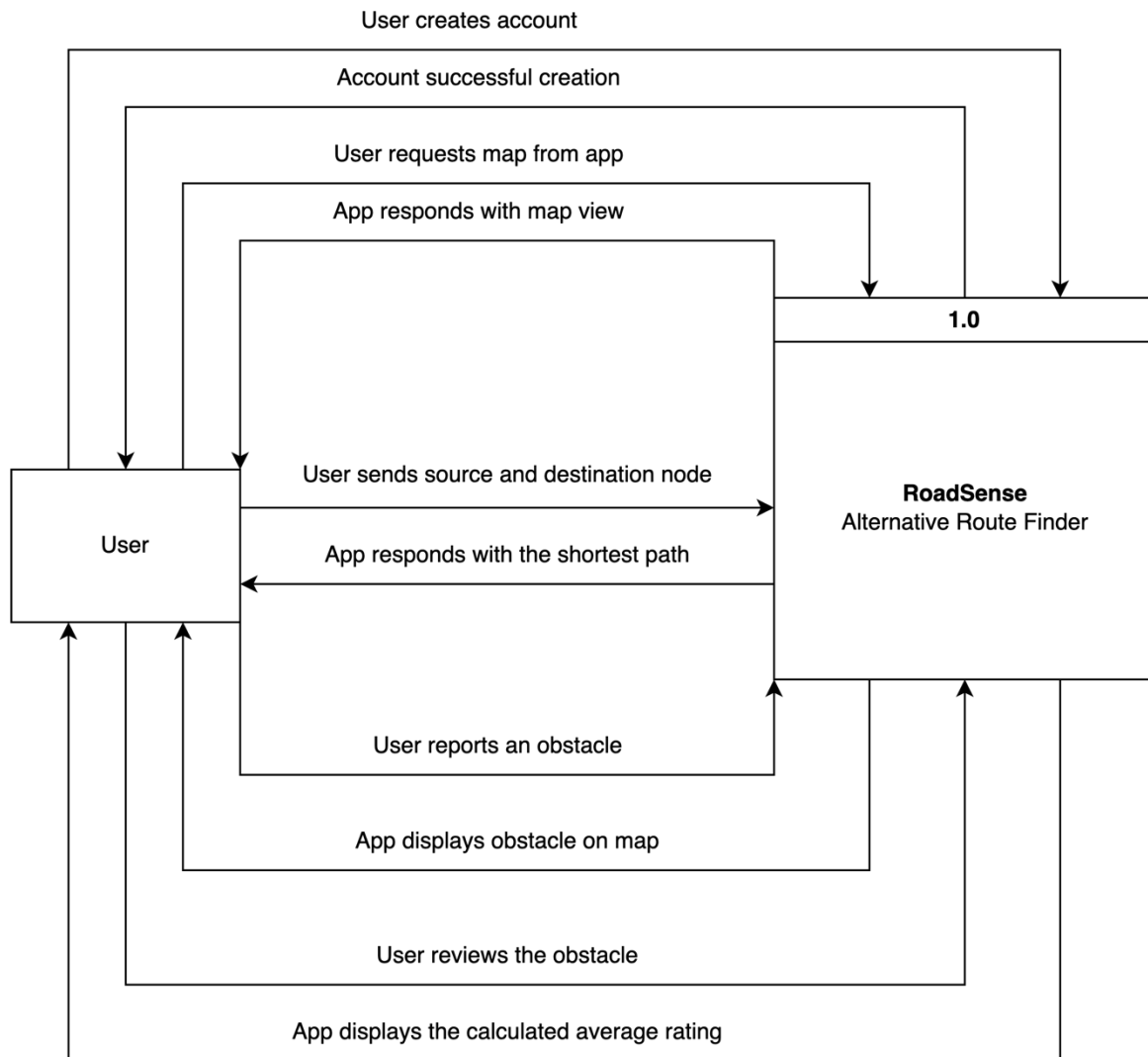


Figure 3.4 Data Flow Diagram Level 0 of RoadSense

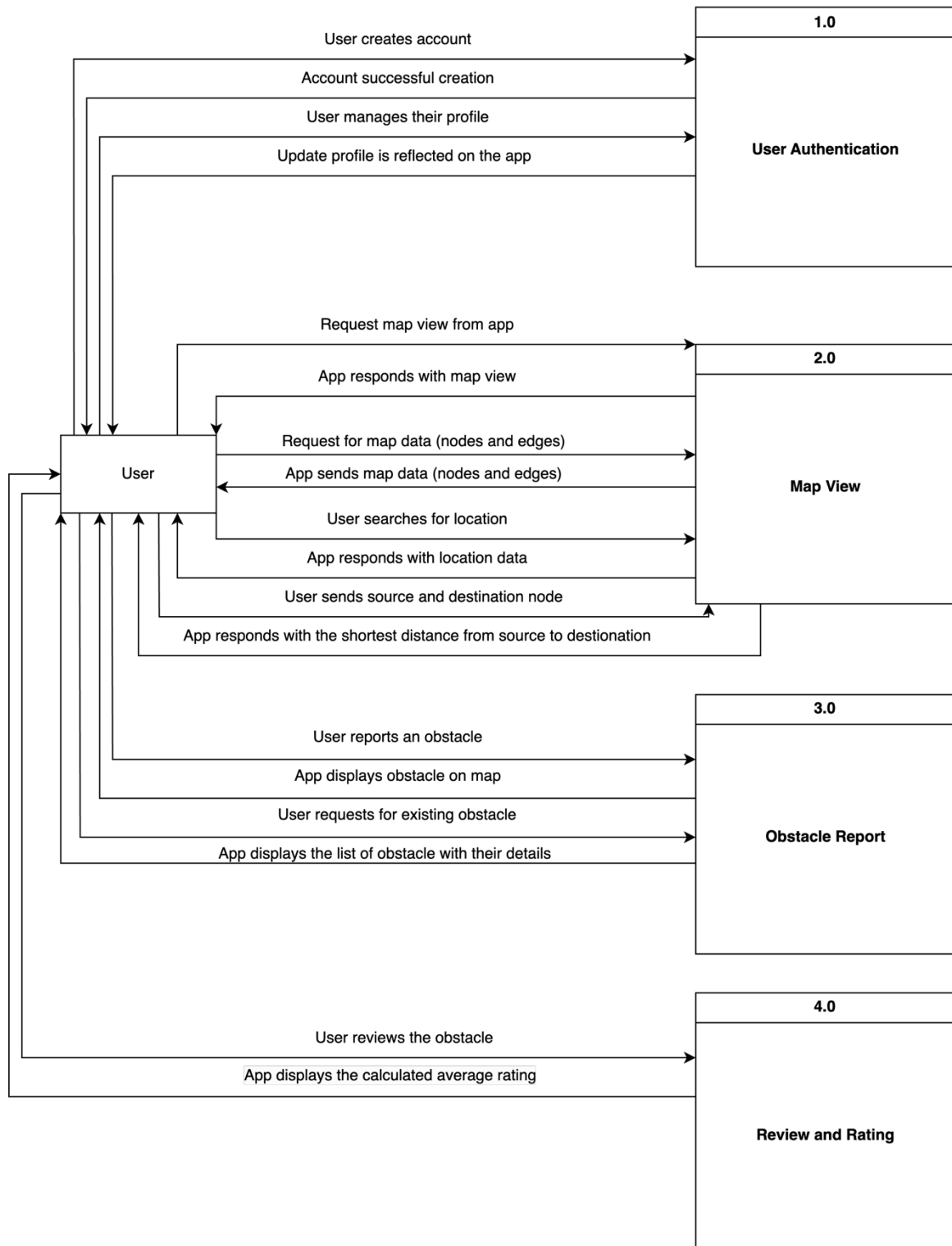


Figure 3.5 Data Flow Diagram Level 1 of RoadSense

3.2 System Design

3.2.1 Architectural Design

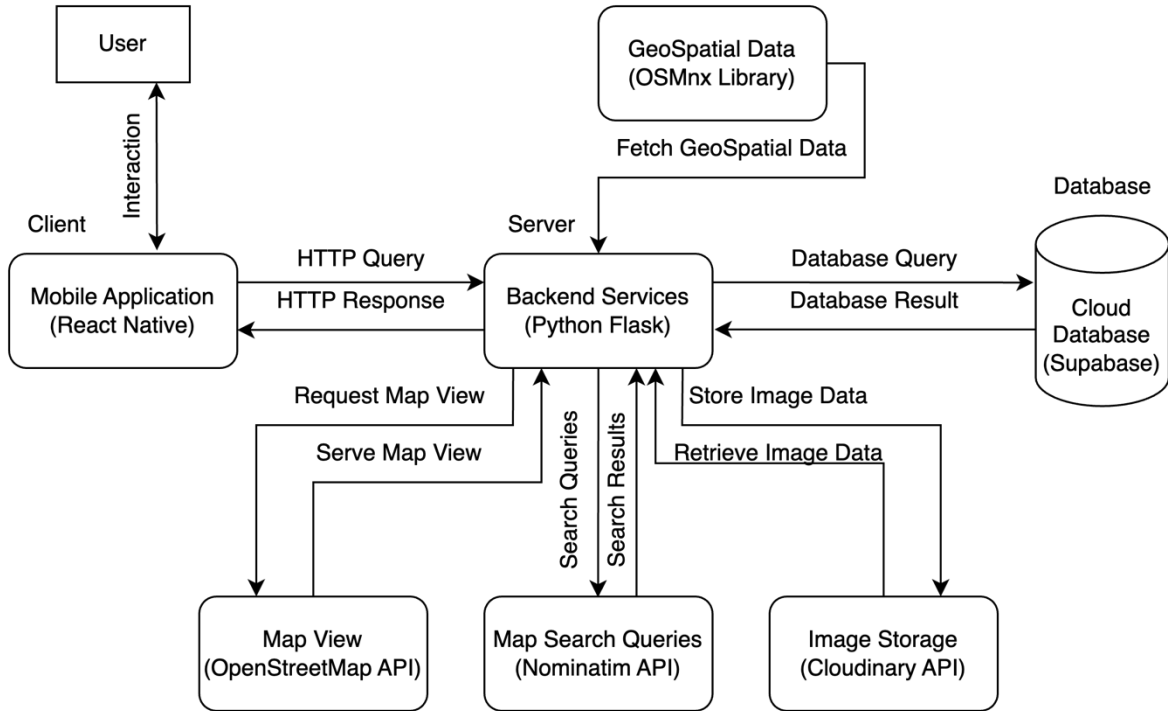


Figure 3.6 Architectural Design of RoadSense

In figure 3.6, the algorithm runs on the backend services, which does all the calculations of the nodes, the flow of data inside the algorithm can be explained further with the help of the diagram of algorithm architecture design as shown below.

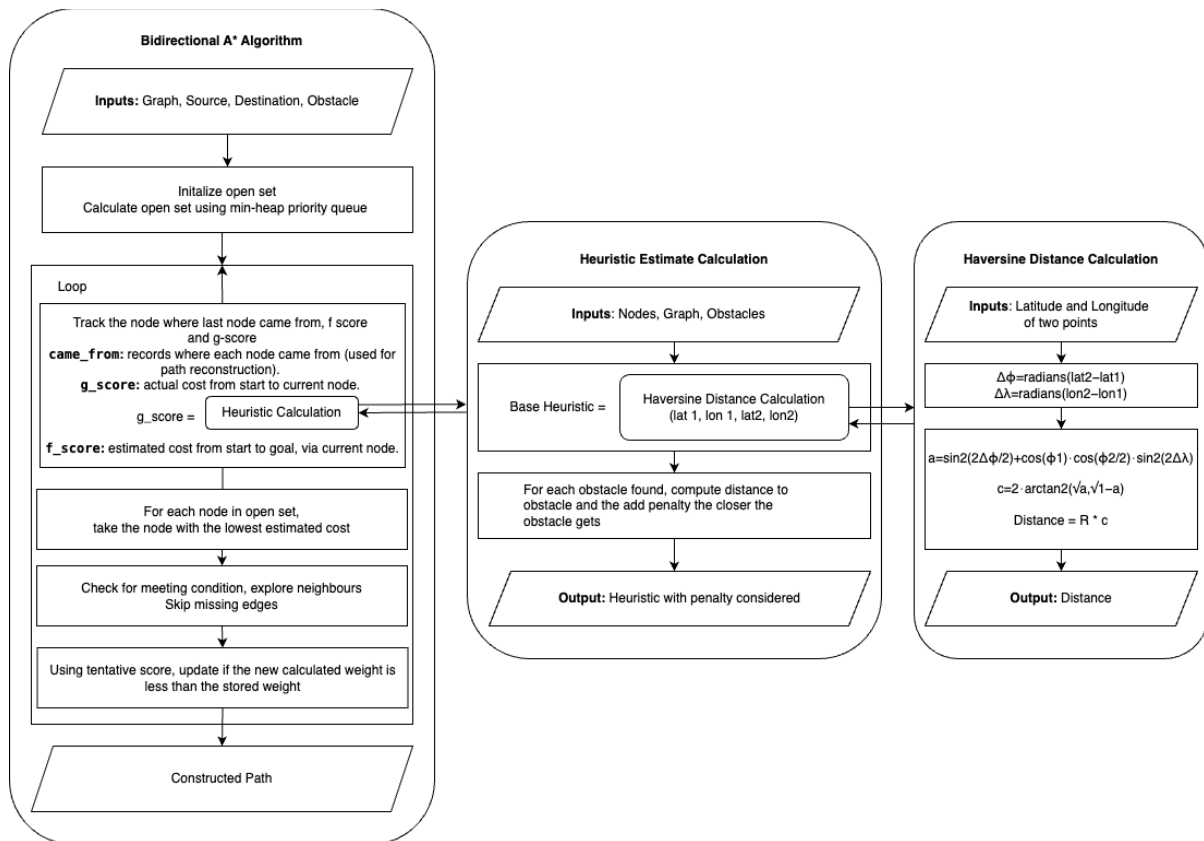


Figure 3.7 Working flow of the Bidirectional A* Algorithm

Bidirectional A* Terms:

Open set: A priority queue (typically a min-heap) of nodes to be explored.

came_from: A dictionary or map recording the preceding node for each explored node, used to reconstruct the final path.

g_score: The actual cost from the starting point to a given node.

f_score: The estimated total cost from start to goal through a node ($f_score = g_score + \text{heuristic estimate}$).

Heuristic Calculation: An estimate of the remaining distance to the goal, adjusted by penalty if close to obstacles.

Heuristic Estimate Terms:

Base Heuristic: The basic distance between two nodes without considering obstacles. This is calculated using the **Haversine Distance**.

Penalty: A cost added to the heuristic value if a node is close to an obstacle (used to steer the path away from it).

lat1, lon1, lat2, lon2: Latitude and longitude of the two points between which the heuristic is being calculated.

Haversine Distance Terms:

$\Delta\phi$ (Delta Phi): Difference in latitudes in radians ($\text{lat2} - \text{lat1}$).

$\Delta\lambda$ (Delta Lambda): Difference in longitudes in radians ($\text{lon2} - \text{lon1}$).

R: Radius of the Earth (typically 6,371 km).

a: Intermediate value in the Haversine formula.

c: Central angle between the two points.

Distance: Final result of the Haversine formula, gives the great-circle distance between two points on a sphere.

3.2.2 Database Schema Design



Figure 3.8 Schema Diagram of RoadSense

3.2.3 Interface Design

Sign up and Sign in Page



Sign In

Sign In

[Don't have an account? Sign Up](#)

 Explore

 Settings

 Roadblock



Sign Up

Sign In

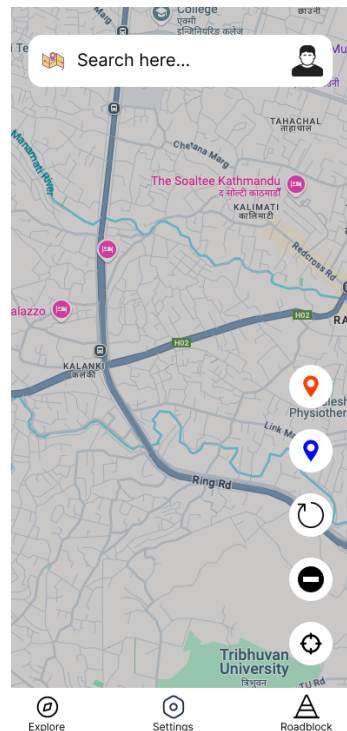
[Don't have an account? Sign Up](#)

 Explore

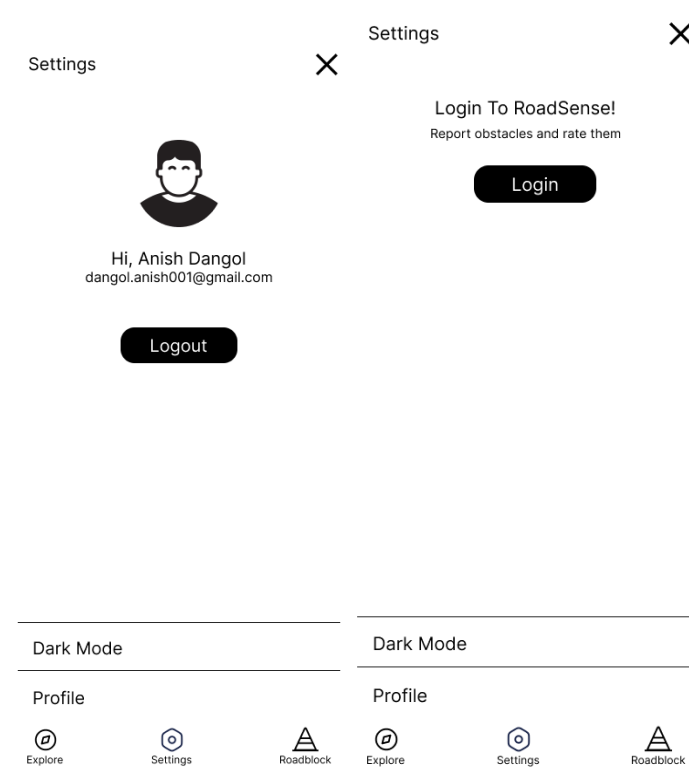
 Settings

 Roadblock

Map View



Default and Logged View



Obstacle List

Obstacles

1. Roadblock at Kalimati
2. Flooding at Kapan
3. Protest at Maitighar
4. Debris at Balaju

Components Design

 Search here...



 Explore

 Settings

 Roadblock

Login To RoadSense!
Report obstacles and rate them

Login

3.3 Algorithm Details

1. Bidirectional A* Algorithm

Bidirectional A* is a pathfinding algorithm designed to efficiently find the shortest path between two points in a graph by simultaneously running two A* searches:

- One forward from the source node.
- One backward from the destination node.

The algorithm stops when both searches meet, significantly reducing the number of nodes visited compared to a single A* search.

Algorithm 1: Bidirectional A*

Inputs: Graph - List of all the available nodes

Source - (Latitude, Longitude) coordinates

Destination - (Latitude, Longitude) coordinates

Obstacles - (Latitude, Longitude) coordinates for obstacles

Outputs: Path - List of nodes from source to destination

Explored Edges - List of Tuples

Process:

1. Check if the source or destination is an obstacle; if so, return None.
2. Initialize two priority queue: `open_set_forward` (starting from Source), `open_set_backward` (starting from Destination).
3. Initialize dictionaries for both directions: `came_from`, `g_score`, `f_score`.
4. Set initial values:
`g_score_forward[source] = 0`
`g_score_backward[destination] = 0`
`f_score_forward[source] = g_score + heuristic(source, destination)`
`f_score_backward[destination] = g_score + heuristic(destination, source)`
5. While both open sets are not empty:
 - a. Pop the node with the lowest `f_score` from each direction:
`current_fwd = open_set_forward.pop()`
`current_bwd = open_set_backward.pop()`

6. If `current_fwd` is in `came_from_backward` or `current_bwd` is in `came_from_forward`, set `meeting_node` and break.
7. For each neighbor of the current node:
 - a. Skip if it's an obstacle or has no valid edge.
 - b. Calculate tentative `g_score`: `tentative_g = g_score[current] + haversine(current, neighbor)`
8. If `tentative_g < g_score[neighbor]`:
 - a. Update:
 - `came_from[neighbor] = current`
 - `g_score[neighbor] = tentative_g`
 - `h = heuristic(neighbor, goal)`
 - `f_score[neighbor] = tentative_g + h`
 - b. Add neighbor to open set if not already present.
9. If no meeting node is found, return `None` and `explored_edges`.
10. Reconstruct the path from source to meeting node using `came_from_forward`.
11. Reconstruct the path from destination to meeting node using `came_from_backward`.
12. Return the full reconstructed path and the list of explored edges.

2. Haversine Distance Algorithm

The Haversine formula is a well-known mathematical method used to calculate the shortest distance between two points on the surface of a sphere, given their latitude and longitude. In practical terms, it's widely used in GPS navigation, mapping applications, and geolocation services to measure distances between two places on Earth.

Algorithm 2: Haversine Distance

Inputs: Latitude and Longitude of Source Node (`lat1`, `lon1` – Coordinates of point 1)

Latitude and Longitude of Destination Node (`lat2`, `lon2` – Coordinates of point 2)

Output: Distance in kilometers between the points

Process:

1. Set Earth's radius `R` to 6371 km.
2. Convert latitudes and longitudes from degrees to radians:
 - $\phi_1 = \text{radians}(\text{lat1})$

$$\phi_2 = \text{radians}(\text{lat}_2)$$

$$\Delta\phi = \text{radians}(\text{lat}_2 - \text{lat}_1)$$

$$\Delta\lambda = \text{radians}(\text{lon}_2 - \text{lon}_1)$$

3. Apply the Haversine formula:

$$a = \sin^2(\Delta\phi / 2) + \cos(\phi_1) \times \cos(\phi_2) \times \sin^2(\Delta\lambda / 2)$$

$$c = 2 \times \text{atan2}(\sqrt{a}, \sqrt{1 - a})$$

$$\text{distance} = R \times c$$

4. Return the distance

3. Heuristic Function

In pathfinding algorithms like A* or Bidirectional A*, the heuristic function estimates the cost from a given node to the goal. This estimate helps guide the search in the right direction and can greatly improve efficiency.

This specific heuristic function does two things:

1. Calculates the straight-line (Haversine) distance between two nodes.
2. Adds a penalty if there are obstacles nearby, making the algorithm more cautious around those areas.

Algorithm 3: Heuristic Function

Inputs: Node ID for Node 1 and Node 2 (node1_id, node2_id – Identifiers of the nodes)

Graph - Contains node positions (lat, lon)

Obstacles - List of obstacle coordinates

Output: Estimated Distance (in kilometers) increased by penalty for nearby obstacles

Process:

1. Retrieve the coordinates of node1 and node2 from the graph.
2. Calculate the base heuristic using the Haversine distance:

$$\text{base_heuristic} = \text{haversine}(\text{node1.lat}, \text{node1.lon}, \text{node2.lat}, \text{node2.lon})$$
3. Initialize obstacle penalty to 0.
4. For each obstacle in the obstacle list:
 - a. Skip if the obstacle is not in the graph.
 - b. Calculate distance from node1 to obstacle using Haversine.
 - c. If distance is less than 0.1 km (100 meters), add a penalty:

d. $\text{obstacle_penalty} += \text{penalty_constant} / \text{distance_to_obstacle}.$

5. Return:

$\text{total_heuristic} = \text{base_heuristic} + \text{obstacle_penalty}$

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Implementation

This part of the documentation emphasizes on how the system was implemented, what type of technologies were used, how the modules were implemented to create the whole system. It is the one of the most important chapters for this documentation as it highlights all the tools and technologies with detailed implementation instruction of this system. This part explains the coding structure, testing details, how the system can be accessed, as well as the whole implementation architecture of this system.

4.1.1 Tools Used

CASE Tools

1. draw.io

draw.io is the leading solutions for web-based sketching and diagramming functionality. You can use our online editor with various storage platforms, and offline with our standalone desktop app. As a security-first diagramming app for teams, we provide the diagramming functionality - you choose where to keep your diagram data. There are many different, including Jira Cloud, Google applications, GitHub, Microsoft applications, Notion and more. Unofficial integrations are available for many other platforms and tools.

2. Project Gantt

Project Gantt is a project management software developed by Microsoft. It's designed to assist project managers in developing plans, assigning resources to tasks, tracking progress, managing budgets, and analyzing workloads. The software provides various features such as Gantt charts, resource management tools, task tracking, and reporting capabilities. Project Gantt is commonly used in industries where complex projects need to be managed, such as construction, engineering, manufacturing, and software development. It helps teams and organizations streamline their project planning and execution processes, leading to more efficient project delivery.

4.1.2 Implementation Details of Modules

This application is divided into five different modules. The modules are as follows:

1. Navigation System

The navigation system has been partially completed for this project, the part where user can find route from source to destination by avoiding obstacles and going through the most optimal part, the remaining part for this module is the real time navigation and turn by turn navigation for the user when they start heading from source to the destination.

2. User Authentication

The user authentication module for this project has been successfully completed, where there is authentication of user information. The sign-up form contains the detailed information of the user with the two-step verification on the mobile phone number to prevent users from making spam accounts and false reporting obstacles which is done using Supabase Auth.

3. Obstacle Report

The obstacle report part of the application is completed, users can report obstacles with the details of the obstacles and the expected duration and when the expected duration is exceeded the obstacle is removed from the map view. Obstacle report makes it easier to find alternative routes for the application which serves as the core functionality of the application.

4. Alternative Route Finder

Alternative route-finding module is also completed for the application. The system implements bidirectional A* algorithm to search for the optimal route and implements a penalty system while searching the nearest nodes to avoid obstacle and provide an alternative path as a result. This helps the users find an alternative road to reach from their source to the destination.

5. User Report Rating

This module ensures the authenticity and verification of obstacles set by users. It allows other users to rate and comment on obstacles, helping to confirm whether a given obstacle genuinely exists in reality. The feedback and interaction mechanisms have been fully implemented, enabling a reliable community-driven verification process.

4.2 Testing

4.2.1 Test Cases for Unit Testing

Table 4.1 User Authentication Test Case

S.N.	Action	Inputs	Expected Outcome	Actual Output	Test Result
1	Leaving Input Field for Registration Empty	Blank	Error Message “All fields are required”	Error Message “All fields are required”	Pass
2	Invalid User	Email: xyz@gmail.com Password: asdf	Error Message “Invalid Login Credentials”	Error Message “Invalid Login Credentials”	Pass
3.	Existing User Details	Email: dangol.anish001@gmail.com Password: anish123	Error Message “User already registered”	Error Message “User already registered”	Pass
4.	Invalid length of phone number	Phone: 981241294182394	Error Message “Invalid Phone Number”	Accepts phone number	Fail

Table 4.2 Map View Test Case

S.N.	Action	Inputs	Expected Outcome	Actual Output	Test Result
1	Long Distance between Source and Destination	Source in West of Map and	Proper Route from	The route sometimes goes through the nodes	Fail

		Destination at East of Map	source to destination	and create a straight line	
2	Adding source and destination in a optimal route	Source and Destination close to each other	Proper Route from source to destination	Proper Route from source to destination	Pass
3	Choosing a node outside of the set map boundary	Choosing source outside of the boundary	Show Error “Out of boundary”	Chooses the closes node of the boundary	Fail
4	Choosing a point at map where no nodes are present	Setting source at a point without node	Choose the nearest node from the point	Choose the nearest node from the point	Pass

Table 4.3 Obstacle Verification Test Case

S.N.	Action	Inputs	Expected Outcome	Actual Output	Test Result
1	Verifying by obstacle	Pressing Verify Button	Obstacle should be verified	Obstacle is verified	Pass
2	Disputing the obstacle right after verifying it	Pressing Dispute button right after Verify	Obstacle verification should be on cooldown	Obstacle Verification is on cooldown	Pass
3	Admin Verifies the Obstacle	Verify Button from Admin	Obstacle should change to verified	Obstacle is changed to verified	Pass

4	Admin Removes the Obstacle	Remove button from admin	Obstacle is removed from the map	Obstacle is removed from the map	Pass
---	----------------------------	--------------------------	----------------------------------	----------------------------------	------

Table 4.4 Obstacle Report Test Case

S.N.	Action	Inputs	Expected Outcome	Actual Output	Test Result
1	Leaving input field blank	Blank	Error “All fields except comments are required”	Error “All fields except comment are required”	Pass
2	Showing obstacle icon based on type and severity	Setting an obstacle with type and severity	Show obstacle with set type and severity	Show obstacle with set type and severity	Pass
3	Upload image of the obstacle	Uploading image of an obstacle	Photo Updated	No photo found	Fail
4	Inputting all fields of the obstacle	Name, Type, Duration, Severity, Comments, Image	Successfully Uploaded	Successfully Uploaded	Pass

Table 4.5 Algorithm Test Case

S.N.	Action	Inputs	Expected Outcome	Actual Output	Test Result
1	Setting Source and Destination in shorter region	Source and Destination	Should show a route from	Shows a route from source to destination	Pass

			source to destination		
2	Setting Source and Destination in longer region	Source and Destination	Should show a route from source to destination	Shows a route from source to destination	Pass
3	Source and Destination in various areas	Source: Kalimati Destination: Gwarko	Route from Kalimati to Gwarko	Route breaks at Sanepa Ring Road	Fail
4	Source to Destination where obstacles are present	Source, Destination, and Obstacle	App finds another route from source to destination	App finds another route from source to destination	Pass
5	Putting Source or Destination at where obstacle is	Adding source at obstacle	App doesn't allow	App doesn't allow	Pass
6	Source or Destination is kept outside map boundary	Source: Kalimati Destination: Birgunj	App chooses the nearest node within the boundary	App chooses the nearest node within the boundary	Pass
7	Setting source and destination in non-updated areas of map	Source and Destination	Map finds old route	Map goes through non-updated area	Fail

4.2.1 Test Cases for System Testing

Table 4.6 System Test Case

S.N.	Action	Inputs	Expected Outcome	Actual Output	Test Result
1	User opens App	Open app by clicking on the icon	Displays map with unauthenticated Features	Displays map with unauthenticated Features	Pass
2	Setting Source and Destination	Source and Destination	Should show a route from source to destination	Shows a route from source to destination	Pass
3	Navigating to road block route	Toggling to “Roadblock” in bottom bar	Sees a list of all available roadblocks	Sees a list of all available roadblocks	Pass
4	Pressing view Icon of road block list	Pressing view icon	Should pan to the obstacle location on map	Pans to the obstacle location on map	Pass
5	Clicking on obstacle icon on map	Press on obstacle icon	Shows obstacle details	Shows obstacle details	Pass
6	Verifying the obstacle after 2 counts of verification	Verifying obstacle	Obstacle becomes verified	Obstacle becomes verified	Pass
7	Setting source and destination in obstacle filled areas	Source and Destination	Map finds shortest path avoiding obstacle	Map finds shortest path avoiding obstacle	Pass

CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS

5.1 Conclusion

The “Road Sense: Alternative Route Finder” project set out to address the limitations of conventional navigation systems in dynamically changing urban environments, particularly in the context of Kathmandu and Lalitpur. Traditional systems like Google Maps and HERE WeGo often fail to reflect real-time disruptions such as accidents, protests, or road maintenance, leading to inefficient routing and commuter frustration [3]. This project introduced a user-focused mobile application that combines real-time, crowdsourced obstacle reporting with intelligent pathfinding using the Bidirectional A* algorithm. While the project has achieved significant milestones in creating a functional and practical system, some challenges and limitations remain. The geographical coverage is currently limited to Kathmandu and Lalitpur, and full real-time turn-by-turn navigation is not yet implemented. Some user reports can still be unreliable due to the absence of automated validation methods. Nevertheless, the system has proven effective in showcasing the viability of a user-driven, obstacle-aware routing platform for urban mobility.

In conclusion, this project represents a successful attempt to bridge the gap between passive navigation systems and the need for real-time, user-responsive solutions in traffic management. It lays a strong foundation for future enhancements, scalability, and adoption in broader contexts.

5.2 Lessons Learnt / Outcome

The development of this system provided valuable insights, both technical and practical, which are summarized below:

1. Importance of User-Generated Data in Navigation

One of the most impactful realizations was how powerful and effective user-reported data can be when integrated into real-time systems. The inclusion of firsthand road condition reports enabled the system to react more swiftly and accurately to dynamic changes in traffic compared to traditional navigation platforms.

2. Iterative Development Encourages Flexibility and Refinement

Adopting the Incremental Model helped mitigate the risks associated with unclear or evolving requirements. Each module could be independently designed, implemented, and tested, allowing room for feedback and refinement. This method also helped manage complexity and ensured stable progress throughout development.

3. Algorithmic Accuracy and Penalty Integration

Implementing the Bidirectional A* algorithm with heuristic penalties for nearby obstacles was crucial in generating practical alternative routes. It highlighted the importance of carefully crafting pathfinding logic to adapt not only to static map data but to real-world, variable inputs.

4. Crowd Validation Enhances Trustworthiness

Allowing users to rate reported obstacles added a layer of credibility and community involvement. This participatory approach encourages data accuracy and discourages misuse, although a need for more robust verification methods remains.

5. Usability Testing Reveals Edge Cases

Thorough testing uncovered various edge cases such as obstacle positioning at start or destination points, boundary issues, and the need for clearer error handling. These findings emphasized the necessity of ongoing user testing and feedback collection.

5.3 Future Recommendations

In light of the project's achievements and observed limitations, the following recommendations are proposed for future enhancements and broader deployment:

1. Expand Geographical Scope Beyond Kathmandu and Lalitpur

While this project focused on the Ring Road of Kathmandu and Lalitpur, traffic congestion is a widespread issue across Nepal's urban areas. The system should be extended to cover additional cities such as Pokhara, Bharatpur, Biratnagar, and Butwal. Eventually, it could be adapted for other developing cities with similar traffic dynamics across South Asia.

2. Implement Real-Time Turn-by-Turn Navigation

One of the most critical upgrades would be adding live navigation features that provide spoken directions, dynamic rerouting, and real-time progress updates. This would require tight integration with the GPS system and route recalculation logic for on-the-go adjustments.

3. Introduce Notification and Alert System

Users should be alerted about nearby obstacles, newly reported disruptions, or changes to previously known routes. Push notifications can improve user awareness and decision-making, especially during high-traffic hours or emergencies [10].

4. Develop More Reliable Obstacle Verification Techniques

The current system relies solely on community ratings to validate obstacle reports. This could be enhanced with additional verification methods such as:

- Image recognition or AI-based analysis of uploaded photos [4].
- Cross-verification based on multiple independent reports.
- Time-based decay for outdated or unconfirmed reports.

5. Optimize Offline Capabilities

Nepal's inconsistent internet coverage makes offline functionality essential. Caching maps, storing recent routes, and allowing offline obstacle reports (to be synced later) can make the application more reliable in low-connectivity areas.

6. Enhance User Experience and Interface Design

Based on feedback, additional improvements can be made to the UI to make the map interaction more intuitive, streamline report submissions, and improve error messages. Accessibility features and local language support would also make the app more inclusive.

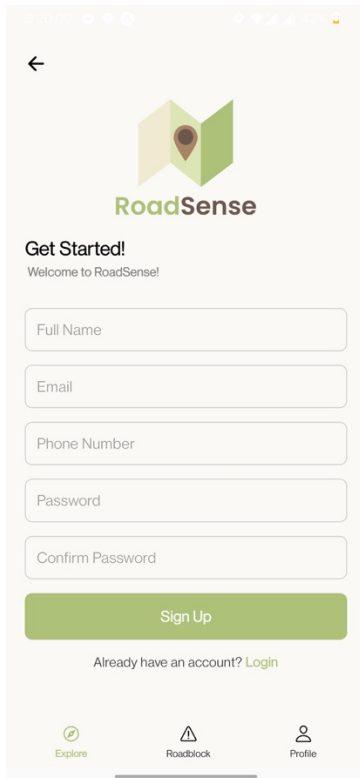
REFERENCES

- [1] J. Adler, Investigating the Learning Effects of Route Guidance and Traffic Advisories on Route Choice Behavior, Troy, NY, USA: Transportation Research Part C, Elsevier, 2001
- [2] P. G. S. S. Harshith Kumar R, Advanced Traffic Navigation System, Bangalore, India : IRE Journals, 2024.
- [3] S. Z. Simeon Nedkov, Enabling Obstacle Avoidance for Google Maps' Navigation Service, Delft, The Netherlands: OTB Research Institute for the Built Environment, TU Delft.
- [4] M. D. a. M. D. Antonios Chatzisavvas *, Optimizing Mobile Robot Navigation Based on A-Star Algorithm for Obstacle Avoidance in Smart Agriculture, 2024.
- [5] L. C. M. E. Youssef Benmessaoud, Real-Time Self-Adaptive Traffic Management System for Optimal Vehicular Navigation in Modern Cities, Tangier, Morocco : Abdelmalek Essaadi University.
- [6] F. R. M. C. S. Oki Ari Saputra, Comparison Analysis of Google Maps, Wisepilot, and Here Wego With User-Centered Design (UCD), Malang, Indonesia: IEEE, 4th International Symposium on Geoinformatics, 2018.
- [7] P. O. V. d. M. A. C. V. J. M. A. J. S. A. A. L. Thiago H. Silva, Traffic Condition Is More Than Colored Lines on a Map: Characterization of Waze Alerts, Belo Horizonte, Brazil / Redmond, USA / France: Springer, in Social Informatics (SocInfo), Lecture Notes in Computer Science (LNCS) 8238, 2013.
- [8] P. K. P. L. Heeket Mehta, Google Maps, International Journal of Computer Applications (IJCA): Mumbai, India, 2019.
- [9] D. D. A. V. G. R. F. W. Ittai Abraham, Alternative Routes in Road Networks, Silicon Valley, CA, USA: ACM Journal of Experimental Algorithmics (via Microsoft Research), 2010.
- [10] R. K. R. S. Gokila S, Smart Traffic: Traffic Congestion Reduction by Shortest Route Search Algorithm, International Journal of Engineering Trends and Technology, 2023.

- [11] S. H. N. G. ., D. S. M. Rhifky Wayahdi1, Greedy, A-Star, and Dijkstra's Algorithms in Finding Shortest Path, International Journal of Advances in Data and Information Systems, April 2021.

APPENDICES

Auth Screen



←


RoadSense

Get Started!
Welcome to RoadSense!

Full Name

Email

Phone Number

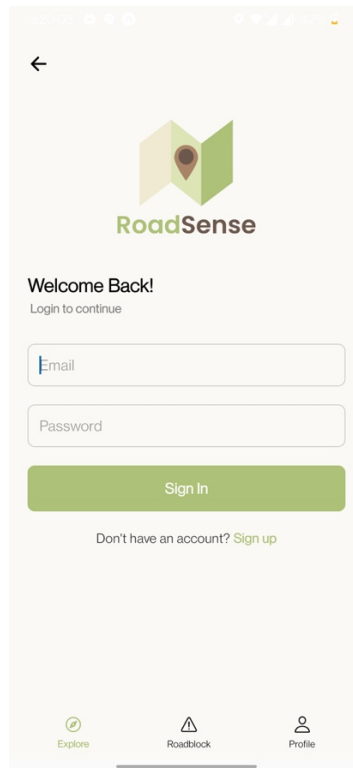
Password

Confirm Password

Sign Up

Already have an account? [Login](#)

 Explore  Roadblock  Profile



←


RoadSense

Welcome Back!
Login to continue

Email

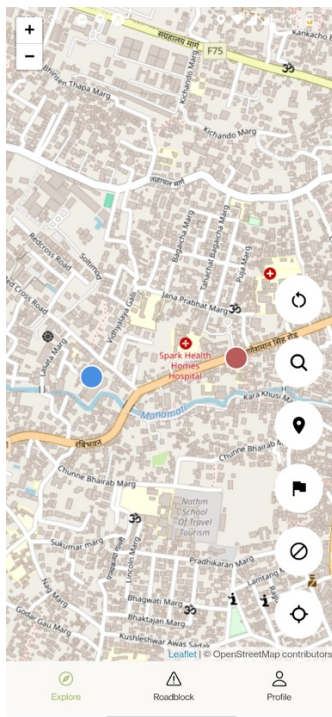
Password

Sign In

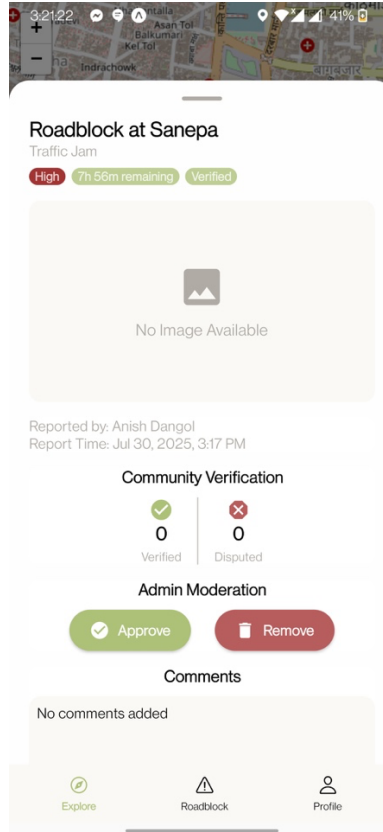
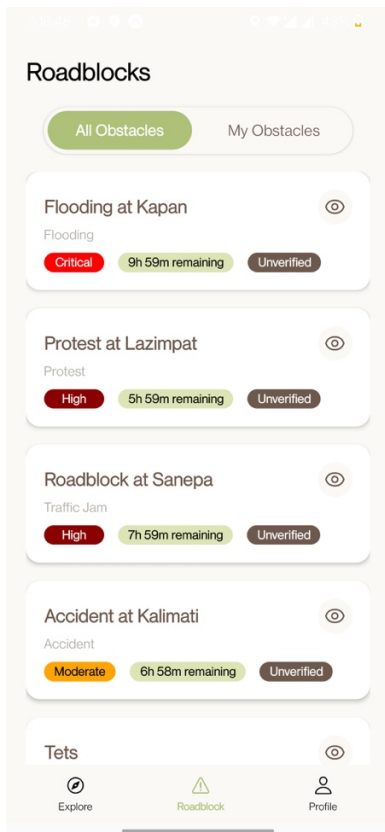
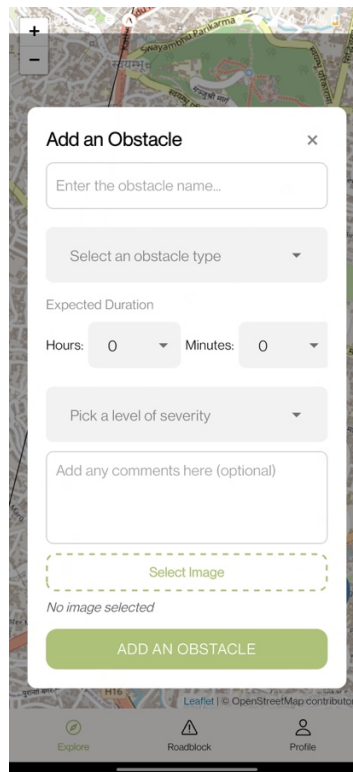
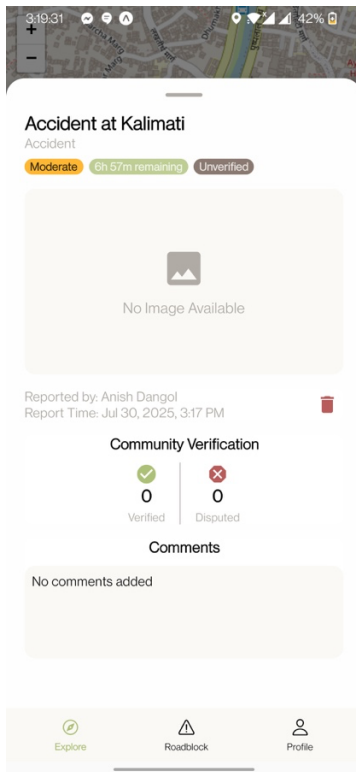
Don't have an account? [Sign up](#)

 Explore  Roadblock  Profile

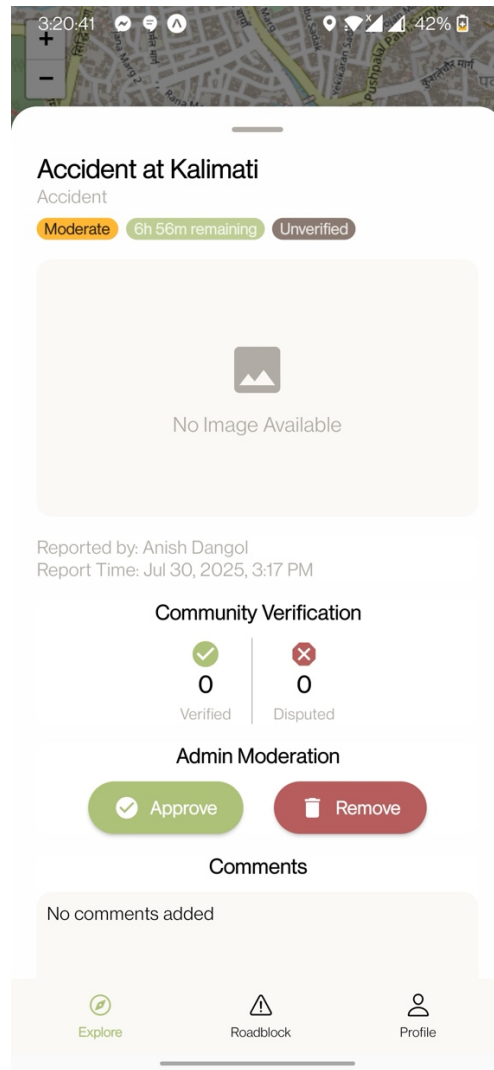
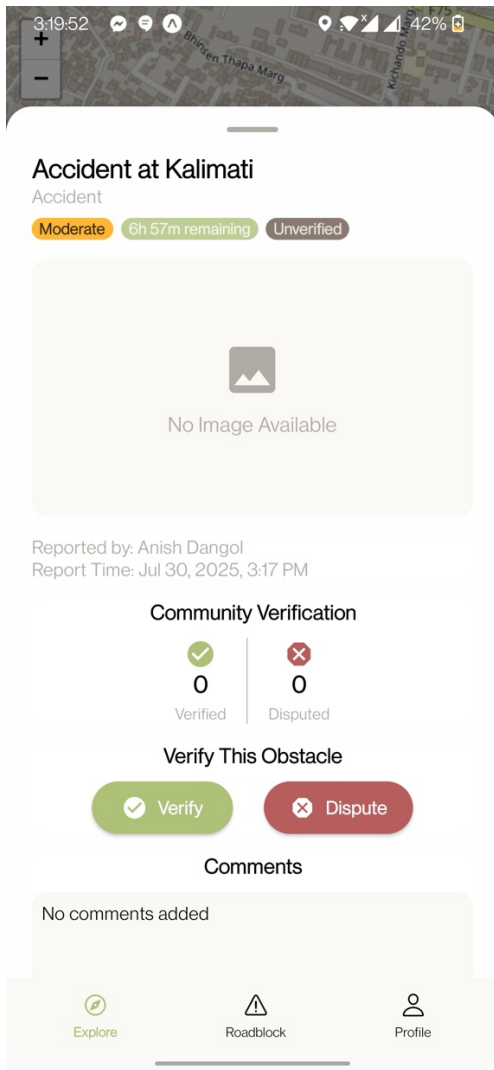
Map Screen



Obstacle Screen



User and Admin Screen on Obstacle



Profile and Search Screen

